2025



Logic Design

GROUP FINAL PROJECT

| TSHWANE UNIVERSITY OF TECHNOLOGY |

TABLE OF CONTENTS	*PAGE*
Practical Name:	*Page 2*
Objective/Aim:	*Page 2*
Material Used:	*Page 2*
Procedure:	*Page 2 – 6*
Practical Layout:	*Page 6 - 7*
Results (Data and Observation):	*Page 7*
Challenges:	*Page 7*
Conclusion:	*Page 8*

Practical Name

Smart Camera Surveillance System using ESP32-CAM and PIR Sensor

Objectives/Aim

The objective of this project is to develop a motion-triggered surveillance system using an ESP32-CAM and a PIR (Passive Infrared) sensor. The system detects motion in its field of vision and automatically captures an image, which is stored on a microSD card. This setup demonstrates how embedded logic and sensor integration can be applied to intelligent security surveillance and data tracking. The project also aims to test and troubleshoot the system for motion detection, image capture, and storage reliability.

Material Used

Component	Quantity	Purpose
ESP32-CAM Module	1	Microcontroller with camera and Wi-Fi
PIR Motion Sensor	1	Detects movement and triggers the camera
Jumper Wires	6+	Connect Components
Breadboard	1	Prototyping layout
USB to TTL Programmer	1	Flashing ESP32-CAM
5V 2A Power Supply	1	Stable power for ESP32-CAM
LED (onboard GPIO 4)	1	Visual feedback on motion detection
Micro SD	1	Store captured image

Procedure

1. **Hardware Setup**
    - Connect the PIR sensor to the ESP32-CAM:
    - VCC → 5V
    - GND → GND
    - OUT → GPIO 13
    - Connect the FTDI programmer to the ESP32-CAM for flashing:
    - TX → U0R (GPIO 3)
    - RX → U0T (GPIO 1)
    - 5V → 5V
    - GND → GND

- GPIO 0 → GND (to enter programming mode)
- Insert a formatted microSD card into the ESP32-CAM slot.
- Use jumper wires to create tidy and secure connections on the breadboard.
- Disconnect FTDI after flashing and power the ESP32-CAM using a 5V C-type cable.

2. **Software Setup**
   - Launch the Arduino IDE from the desktop.
   - Install and configure the ESP32 board package via the Board Manager.
   - If required, add the ESP32 board URL under File > Preferences, then restart the IDE.
   - Select **AI Thinker ESP32-CAM** under Tools > Board > ESP32 Arduino.
   - Create and save a new sketch file with a clear name (e.g., Group3_Project.ino).
   - Select the correct COM port for the FTDI programmer under Tools > Port.
   - Use the Library Manager to install required libraries such as esp_camera and SD_MMC.

3. **Code Uploading and Testing**
   - Measure baseline distance at startup.
   - Upload the sketch and monitor initialization via Serial Monitor (baud rate: 115200).
   - Recompile and test the system to ensure consistent operation.
   - Observe motion detection and image capture logs in the Serial Monitor.

ARDUINO CODE:

```
#include "esp_camera.h"

#include "SD_MMC.h"

#define PIR_PIN 13

void setup() {

Serial.begin(115200);

pinMode(PIR_PIN, INPUT);

// Camera config

camera_config_t config;

config.ledc_channel = LEDC_CHANNEL_0;

config.ledc_timer = LEDC_TIMER_0;

config.pin_d0 = Y2_GPIO_NUM; config.pin_d1 =
```

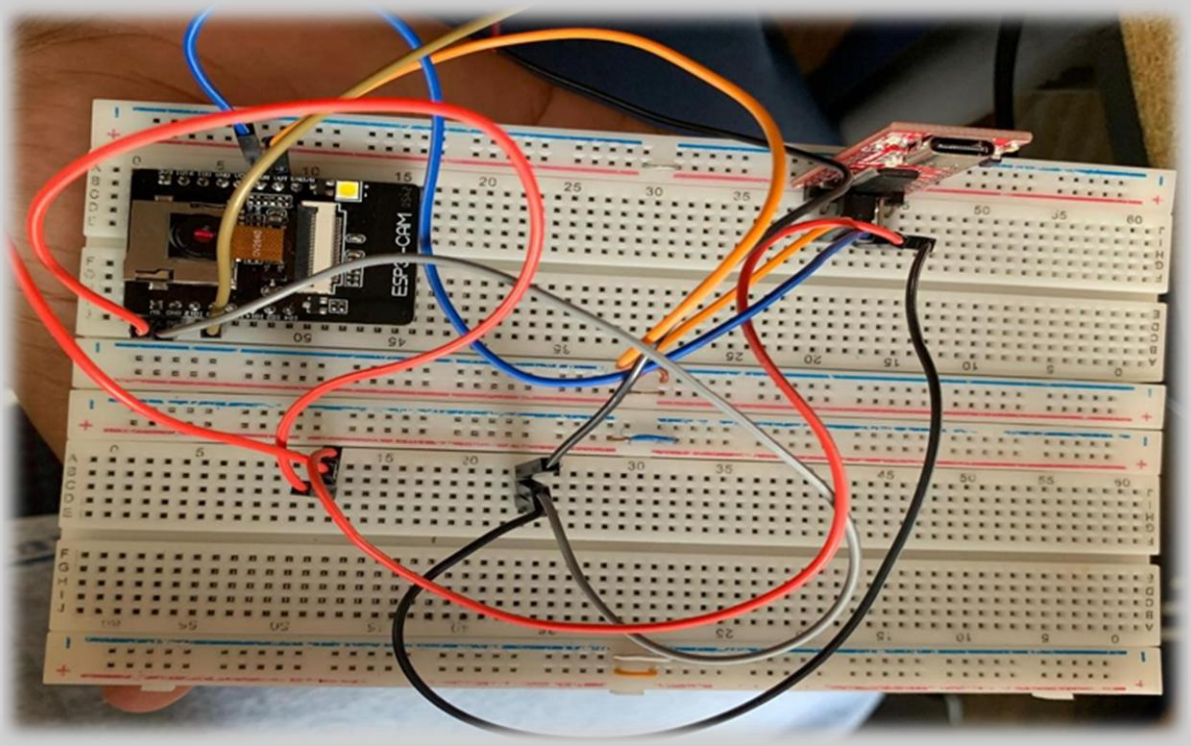
```
Y3_GPIO_NUM; config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM; config.pin_d4 =
Y6_GPIO_NUM; config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM; config.pin_d7 =
Y9_GPIO_NUM; config.pin_xclk =
XCLK_GPIO_NUM; config.pin_pclk =
PCLK_GPIO_NUM; config.pin_vsync =
VSYNC_GPIO_NUM; config.pin_href =
HREF_GPIO_NUM; config.pin_sscb_sda =
SIOD_GPIO_NUM; config.pin_sscb_scl =
SIOC_GPIO_NUM; config.pin_pwdn =
PWDN_GPIO_NUM; config.pin_reset =
RESET_GPIO_NUM; config.xclk_freq_hz =
20000000; config.pixel_format =
PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_UXGA;
config.jpeg_quality = 10;
config.fb_count = 1;
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
Serial.printf("Camera init failed with error 0x%x", err);
return;
}
Serial.println("Camera initialized");
// SD Card init
if(!SD_MMC.begin()){
Serial.println("SD Card Mount Failed");
return;
```

```
}
uint8_t cardType = SD_MMC.cardType();
if(cardType == CARD_NONE){
Serial.println("No SD Card attached");
return;
}
Serial.println("SD Card initialized");
}
void loop() {
int motion = digitalRead(PIR_PIN);
if (motion == HIGH) {
Serial.println("Motion detected!");
camera_fb_t * fb = esp_camera_fb_get();
if(!fb) {
Serial.println("Camera capture failed");
return;
}
// Save to SD
String path = "/picture" + String(random(0,100)) + ".jpg";
File file = SD_MMC.open(path.c_str(), FILE_WRITE);
if(!file){
Serial.println("Failed to open file in writing mode");
} else {
file.write(fb->buf, fb->len);
Serial.printf("Saved file to path: %s\n", path.c_str());
}
file.close();
esp_camera_fb_return(fb);
```

```
delay(5000); // 5 second cooldown delay
}
delay(100);
}
```

4. **Testing**
   - Simulate motion by placing/removing objects.
   - Observe LED flash and Serial output.

CIRCUIT DIAGRAM:



Practical Layout

- ESP32-CAM and PIR sensor placed side by side, facing the same direction.
- Breadboard used for initial testing; Veroboard used for final stable connections.
- Jumper wires arranged neatly to prevent disconnections.
- MicroSD card inserted directly into ESP32-CAM slot.
- Power supplied via 5V C-type cable.
- Serial Monitor used to observe motion detection and image capture logs.

Results (Data and Observations)

- Motion Detection Range: PIR sensor reliably detected motion within 5–7 meters.

- Image Capture and Storage: Each motion event triggered the camera to capture and save a photo on the microSD card.

- Power Stability: Improved by using Veroboard for VCC and GND lines.

- Timing Observations:

- PIR response time: <1 second

- Image capture delay: 5 seconds between captures

- Code Compilation: Successful with no errors; upload verified via Serial Monitor.

- Serial Output: Confirmed motion detection and image save path

Challenges

- Sensor Availability: Initially planned to use a PIR sensor but considered ultrasonic as an alternative due to availability. Ultimately used PIR for simplicity and reliability.

- Serial Monitor Issues: No data appeared initially, resolved by verifying baud rate and COM port settings.

- Unstable Readings: Early tests showed inconsistent motion detection. Improved by securing wiring and adjusting sensor angle.

- Power Management: ESP32-CAM was sensitive to voltage drops. Using Veroboard improved stability.

- Timing Conflicts: Without a delay, multiple images were captured rapidly. Added a 5-second cooldown to regulate capture frequency.

- Code Upload Errors: Faced issues with flashing, resolved by confirming GPIO 0 was grounded during upload.

Conclusion

Using the ESP32-CAM and PIR sensor, this project successfully created a smart camera surveillance system that is fully operational. The system detects motion and responds by capturing and storing images, demonstrating effective sensor integration and embedded logic design. All objectives were met, and the final build performed reliably during testing. The project also highlighted the importance of teamwork,

troubleshooting, and iterative design in achieving a functional and well-documented solution.